UNIDAD VII

Clases genéricas o parametrizadas

Definición

- Una plantilla (template) es un patrón para crear funciones y clases.

Hay dos tipos de templates:

- Funciones Templates (de tipos parametrizados)
- Clases Templates (clases parametrizadas)

Propósito

- Evitar escribir múltiples versiones de la misma función para llevar a cabo la misma operación sobre datos de distinto tipo.
- Ejemplo función swap() para intercambiar el valor de dos variables
- Los datos podrian ser:
  - int
  - Float
  - Char
  - Fecha
  - Racional
  - …

Funciónes Templates

Una función plantilla tiene *uno o más* parámetros.

```
template<class T>
return-type function-name( T param )
```

- `T` es un parámetro de la plantilla (o template). Este indica el tipo de dato que será suministrado cuando la función sea llamada.

Función swap()

```
void swap(int & X, int & Y)
{
  int temp = X;
  X = Y;
  Y = temp;
}

void swap(float & X, float & Y)
{
  float temp = X;
  X = Y;
  Y = temp;
}
```

- Dos funciones diferentes sobrecargadas (overloaded).
- Son funciones distintas para el compilador dado que la firma está compuesta por el nombre de la función y su lista de parámetros.

Candidato para una plantilla de función

Plantilla para la Función swap()

- Esta puede ser la plantilla para la función swap.
- Esta plantilla trabaja con cualquier tipo de dato que soporte el operador de asignación.

```
template <class TParam>
void Swap( TParam & X, TParam & Y )
{
  TParam temp = X;
  X = Y;
  Y = temp;
}
```

Llamando a swap()

Cuando llamamos a swap(), el único requerimiento es que ambos parámetros sean del mismo tipo y que sean modificables vía el operador de asignación =.

```
int A = 5;
int B = 6;
swap( A, B );

float X=3.1;
float Y=5.8;
swap( X, Y );
```

Llamado a swap()

Por supuesto, algunas combinaciones no funcionan.

No se puede pasar constantes, y no podemos pasar tipos incompatibles.

```
/* casos que no funcionan */
int B = 6;
swap( 10, B );


bool M = true;
swap( B, M );
```

Plantillas para Clases (Class Templates)

- Las plantillas de clases permiten crear nuevas clases con tipos de datos no definidos.

- Usamos plantillas de clases cuando de otra manera estaríamos forzados a crear múltiples clases con los mismos atributos y operaciones.

Declaración de una Plantilla de Clase

Formato general para declarar una plantilla de clases. El parámetro T representa un tipo de datos.

```
template <class T>
class className {
public:

private:

};
```

Clase Vector:

Una clase arreglo para almacenar elementos de cualquier tipo de datos.

```
template <class T>
class vector{
   private:
      T *V;
      int tam;
   public:
    vector(int);
    vector(const vector&);
    vector operator+(vector x);
    vector operator-(vector x);
    void operator=(vector x);
    T& operator[](int i);
    ~vector();
};
```

Clase Vector

Implementacion:

```
template<class T>
vector<T>::vector(int t){
 tam=10;
 V=new T [tam];
}

template<class T>
T &vector<T>::operator[](int i){
   return V[i];
}
```

Uso de la clase vector

Un programa cliente puede crear cualquier tipo de arreglo usando la plantilla mientras la clase admita la asignación (operador =).

```
int main(){
 vector<int> a(10),b(10),c(10);
...
 vector<float> af(10),bf(10),cf(10);
...
 vector<racional> r1(10),r2(10),r3(10);

}
```